DESARROLLO DE APLICACIONES CON OBJETOS · TPI · TEMA 3

Persistencia con SQLite

Los ejercicios del repositorio de la cátedra resueltos: ABM de personas y reportes con SQL desde Python

SQLite en una hoja | ABM completo | 8 reportes con SQL | Base de la cátedra

Qué hay en esta guía

Los dos ejercicios del **tema 3** del repositorio de la cátedra (2026-guias): pasar el ABM de personas a una base **SQLite** y el sistema completo de gestión de personas con reportes. Se resuelven juntos en un solo programa, probado con la base `personas.sqlite` de la cátedra (1986 personas). Es la base de lo que se usa en el **trabajo integrador**: guardar y consultar los datos en una base.

0 SQLite desde Python, en una hoja

Paso	Código	Para qué
Conectar	`conexion = sqlite3.connect("personas.sqlite")`	Abre el archivo de la base (si no existe, lo crea vacío)
Filas por nombre	`conexion.row_factory = sqlite3.Row`	Leer `fila["apellido"]` en lugar de `fila[2]`
Consultar	`conexion.execute("SELECT … WHERE documento = ?", (doc,))`	Los valores van con `?`, nunca pegados al texto (evita la inyección SQL)
Leer	`.fetchone()` · `.fetchall()`	Una fila (o `None`) · todas en una lista
Modificar	`with conexion:` + `INSERT` / `UPDATE` / `DELETE`	Transacción: hace `commit` si todo sale bien y `rollback` si hay un error
¿Cambió algo?	`cursor.rowcount`	Cuántas filas tocó el `UPDATE` o el `DELETE` (0 = no existía)
Cerrar	`conexion.close()`	Al salir del programa

Reporte	SQL
Ordenar	`SELECT * FROM Personas ORDER BY apellido, nombre`
Filtrar	`WHERE edad >= 18` · `WHERE nombre LIKE 'M%'`
Contar y calcular	`SELECT COUNT(*), AVG(edad), MIN(edad), MAX(edad) FROM Personas`
Por rangos	`CASE WHEN edad < 18 THEN '0 a 17' WHEN … END` + `GROUP BY`
Repetidos	`GROUP BY apellido HAVING COUNT(*) > 1`
El más joven	`ORDER BY edad LIMIT 1`

1 Base de personas

Acceso a bases de datos · Solución propuesta

Enunciado (tal cual)

Modificar el ABM de personas en memoria de la unidad 3 para que los datos sean almacenados en la base de datos personas.sqlite

Se resuelve con el programa del ejercicio 2, que hace el ABM completo sobre la misma base.

2 Sistema de gestión de personas

ABM con persistencia · Solución propuesta

Enunciado (tal cual)

Desarrollar un programa en Python que gestione una base de datos de personas utilizando SQLite3.

- **Tabla:** `Personas`
- **Columnas:**
  - `documento` (TEXT, PRIMARY KEY): Número de documento único
  - `nombre` (TEXT): Nombre de la persona
  - `apellido` (TEXT): Apellido de la persona
  - `edad` (INTEGER): Edad actual de la persona

Menú Principal

El programa debe mostrar un menú de consola con las siguientes opciones:

```
=== SISTEMA DE GESTIÓN DE PERSONAS ===
1. Agregar nueva persona
2. Registrar cumpleaños (incrementar edad)
3. Modificar datos de una persona
4. Eliminar persona
5. Buscar persona por documento
6. Reportes
7. Salir
```

Funcionalidades a Implementar

1. Operaciones de Modificación de Datos

a) Agregar nueva persona

- Solicitar: documento, nombre, apellido y edad
- Validar que el documento no exista previamente
- Insertar el registro en la tabla

b) Registrar cumpleaños

- Solicitar el documento de la persona
- Incrementar la edad en 1
- Mostrar mensaje de confirmación con los datos actualizados

c) Modificar datos de una persona

- Permitir cambiar nombre, apellido o edad

- Buscar por documento
- Actualizar solo los campos indicados

d) Eliminar persona

- Solicitar documento
- Confirmar antes de eliminar
- Mostrar mensaje de éxito o error

e) Buscar persona por documento

- Mostrar todos los datos de la persona encontrada

2. Menú de Reportes

```
=== REPORTES ===
1. Listar todas las personas ordenadas por apellido
2. Personas mayores de edad (≥18 años)
3. Personas menores de edad (<18 años)
4. Cantidad de personas por rango de edad
5. Persona más joven y más longeva
6. Estadísticas generales (promedio, mínimo, máximo de edad)
7. Personas cuyo nombre empiece con una letra específica
8. Apellidos con más de una persona (GROUP BY + HAVING)
9. Volver al menú principal
```

Cómo se piensa

- Una función por opción del menú, y cada una abre su consulta con `?` para los datos que escribe el usuario.
- Todo lo que modifica va dentro de `with conexion:`, así se guarda (commit) solo si salió bien.
- Los reportes los calcula la base con SQL (`COUNT`, `AVG`, `GROUP BY`, `HAVING`, `CASE`), no Python recorriendo listas.
- Los listados largos muestran las primeras 10 filas y el total.

principal.py

```python
import sqlite3

BASE = "personas.sqlite"


def conectar():
    conexion = sqlite3.connect(BASE)
    conexion.row_factory = sqlite3.Row          # las filas se leen por nombre de columna
    return conexion


def mostrar(p):
    print(f"   {p['documento']}: {p['apellido']}, {p['nombre']} ({p['edad']} años)")


def buscar(conexion, documento):
    return conexion.execute("SELECT * FROM Personas WHERE documento = ?",
(documento,)).fetchone()


# ---------- Altas, bajas y modificaciones ----------
```

```
def agregar(conexion):
    documento = int(input("Documento: "))
    if buscar(conexion, documento):
        print("   Ya existe una persona con ese documento")
        return
    nombre, apellido = input("Nombre: ").upper(), input("Apellido: ").upper()
    edad = int(input("Edad: "))
    with conexion:                                # commit al terminar bien, rollback si hay
error
        conexion.execute("INSERT INTO Personas (documento, nombre, apellido, edad) VALUES (?, ?,
?, ?)",
                         (documento, nombre, apellido, edad))
    print("   Persona agregada")


def cumpleanios(conexion):
    documento = int(input("Documento: "))
    with conexion:
        cursor = conexion.execute("UPDATE Personas SET edad = edad + 1 WHERE documento = ?",
(documento,))
    if cursor.rowcount == 0:
        print("   No existe esa persona")
    else:
        print("   ¡Feliz cumpleaños!")
        mostrar(buscar(conexion, documento))


def modificar(conexion):
    documento = int(input("Documento: "))
    persona = buscar(conexion, documento)
    if not persona:
        print("   No existe esa persona")
        return
    mostrar(persona)
    print("   (Enter para dejar el dato como está)")
    cambios = {}
    for campo in ("nombre", "apellido", "edad"):
        valor = input(f"{campo.capitalize()} (nuevo): ")
        if valor:
            cambios[campo] = int(valor) if campo == "edad" else valor.upper()
    if not cambios:
        print("   Sin cambios")
        return
    columnas = ", ".join(f"{c} = ?" for c in cambios)          # solo los campos indicados
    with conexion:
        conexion.execute(f"UPDATE Personas SET {columnas} WHERE documento = ?",
(*cambios.values(), documento))
    mostrar(buscar(conexion, documento))


def eliminar(conexion):
    documento = int(input("Documento: "))
    persona = buscar(conexion, documento)
    if not persona:
        print("   No existe esa persona")
        return
    mostrar(persona)
    if input("¿Confirma la eliminación? (s/n): ").lower() == "s":
        with conexion:
            conexion.execute("DELETE FROM Personas WHERE documento = ?", (documento,))
        print("   Persona eliminada")
```

```
    else:
        print("   No se eliminó")


def consultar(conexion):
    persona = buscar(conexion, int(input("Documento: ")))
    mostrar(persona) if persona else print("   No existe esa persona")


# ---------- Reportes ----------
def listar(conexion, sql, parametros=(), maximo=10):
    filas = conexion.execute(sql, parametros).fetchall()
    for p in filas[:maximo]:
        mostrar(p)
    if len(filas) > maximo:
        print(f"   … y {len(filas) - maximo} más")
    print(f"   Total: {len(filas)}")


def reportes(conexion):
    opciones = {
        "1": ("Todas", lambda: listar(conexion, "SELECT * FROM Personas ORDER BY apellido,
nombre")),
        "2": ("Mayores", lambda: listar(conexion, "SELECT * FROM Personas WHERE edad >= 18 ORDER
BY apellido")),
        "3": ("Menores", lambda: listar(conexion, "SELECT * FROM Personas WHERE edad < 18 ORDER
BY apellido")),
        "4": ("Por rango", lambda: rangos(conexion)),
        "5": ("Extremos", lambda: extremos(conexion)),
        "6": ("Estadísticas", lambda: estadisticas(conexion)),
        "7": ("Por letra", lambda: por_letra(conexion)),
        "8": ("Apellidos repetidos", lambda: apellidos_repetidos(conexion)),
    }
    opcion = ""
    while opcion != "9":
        print("REPORTES: " + " · ".join(f"{k} {t}" for k, (t, _) in opciones.items()) + " · 9
Volver")
        opcion = input("Opción: ")
        if opcion in opciones:
            opciones[opcion][1]()


def rangos(conexion):
    sql = """SELECT CASE WHEN edad < 18 THEN '0 a 17'
                         WHEN edad < 30 THEN '18 a 29'
                         WHEN edad < 50 THEN '30 a 49'
                         WHEN edad < 65 THEN '50 a 64'
                         ELSE '65 o más' END AS rango, COUNT(*) AS cantidad
             FROM Personas GROUP BY rango ORDER BY MIN(edad)"""
    for fila in conexion.execute(sql):
        print(f"   {fila['rango']}: {fila['cantidad']}")


def extremos(conexion):
    print("   Más joven:"); mostrar(conexion.execute("SELECT * FROM Personas ORDER BY edad,
apellido LIMIT 1").fetchone())
    print("   Más longeva:"); mostrar(conexion.execute("SELECT * FROM Personas ORDER BY edad
DESC, apellido LIMIT 1").fetchone())


def estadisticas(conexion):
```

```
    f = conexion.execute("SELECT COUNT(*) AS n, AVG(edad) AS prom, MIN(edad) AS mi, MAX(edad) AS
ma FROM Personas").fetchone()
    print(f"   {f['n']} personas · promedio {f['prom']:.2f} · mínima {f['mi']} · máxima
{f['ma']}")


def por_letra(conexion):
    letra = input("Letra: ").upper()[:1]
    listar(conexion, "SELECT * FROM Personas WHERE nombre LIKE ? ORDER BY nombre", (letra +
"%",))


def apellidos_repetidos(conexion):
    sql = """SELECT apellido, COUNT(*) AS cantidad FROM Personas
             GROUP BY apellido HAVING COUNT(*) > 1 ORDER BY cantidad DESC, apellido"""
    filas = conexion.execute(sql).fetchall()
    for f in filas[:10]:
        print(f"   {f['apellido']}: {f['cantidad']}")
    print(f"   {len(filas)} apellidos se repiten")


def main():
    conexion = conectar()
    menu = {"1": ("Agregar", agregar), "2": ("Cumpleaños", cumpleanios), "3": ("Modificar",
modificar),
            "4": ("Eliminar", eliminar), "5": ("Buscar", consultar), "6": ("Reportes", reportes)}
    opcion = ""
    while opcion != "7":
        print("PERSONAS: " + " · ".join(f"{k} {t}" for k, (t, _) in menu.items()) + " · 7 Salir")
        opcion = input("Opción: ")
        if opcion in menu:
            menu[opcion][1](conexion)
    conexion.close()


if __name__ == "__main__":
    main()
```

Salida al correrlo (una prueba de todas las opciones; al final se borra la persona agregada)

```
PERSONAS: 1 Agregar · 2 Cumpleaños · 3 Modificar · 4 Eliminar · 5 Buscar · 6 Reportes · 7 Salir
Opción: 1
Documento: 99000001
Nombre: Lucía
Apellido: Ferreyra
Edad: 20
   Persona agregada
PERSONAS: 1 Agregar · 2 Cumpleaños · 3 Modificar · 4 Eliminar · 5 Buscar · 6 Reportes · 7 Salir
Opción: 1
Documento: 99000001
   Ya existe una persona con ese documento
PERSONAS: 1 Agregar · 2 Cumpleaños · 3 Modificar · 4 Eliminar · 5 Buscar · 6 Reportes · 7 Salir
Opción: 2
Documento: 99000001
   ¡Feliz cumpleaños!
   99000001: FERREYRA, LUCÍA (21 años)
PERSONAS: 1 Agregar · 2 Cumpleaños · 3 Modificar · 4 Eliminar · 5 Buscar · 6 Reportes · 7 Salir
Opción: 3
Documento: 99000001
   99000001: FERREYRA, LUCÍA (21 años)
   (Enter para dejar el dato como está)
```

```
Nombre (nuevo):
Apellido (nuevo):
Edad (nuevo): 25
   99000001: FERREYRA, LUCÍA (25 años)
PERSONAS: 1 Agregar · 2 Cumpleaños · 3 Modificar · 4 Eliminar · 5 Buscar · 6 Reportes · 7 Salir
Opción: 5
Documento: 99000001
   99000001: FERREYRA, LUCÍA (25 años)
PERSONAS: 1 Agregar · 2 Cumpleaños · 3 Modificar · 4 Eliminar · 5 Buscar · 6 Reportes · 7 Salir
Opción: 6
REPORTES: 1 Todas · 2 Mayores · 3 Menores · 4 Por rango · 5 Extremos · 6 Estadísticas · 7 Por
letra · 8 Apellidos repetidos · 9 Volver
Opción: 2
   18856717: AGUILAR, LUIS (89 años)
   19476047: AGUILAR, CARLOS (45 años)
   25276295: AGUILAR, IGNACIO (64 años)
   32063925: AGUILAR, ANTONIA (54 años)
   35827141: AGUILAR, CRISTIAN (33 años)
   36681683: AGUILAR, JAIME (81 años)
   47483597: AGUILAR, ROBERTO (56 años)
   48415614: AGUILAR, JOSE RAMON (74 años)
   51097397: AGUILAR, DOMINGO (69 años)
   55297373: AGUILAR, MARIA CARMEN (68 años)
   … y 1792 más
   Total: 1802
REPORTES: 1 Todas · 2 Mayores · 3 Menores · 4 Por rango · 5 Extremos · 6 Estadísticas · 7 Por
letra · 8 Apellidos repetidos · 9 Volver
Opción: 4
   0 a 17: 185
   18 a 29: 249
   30 a 49: 445
   50 a 64: 321
   65 o más: 787
REPORTES: 1 Todas · 2 Mayores · 3 Menores · 4 Por rango · 5 Extremos · 6 Estadísticas · 7 Por
letra · 8 Apellidos repetidos · 9 Volver
Opción: 5
   Más joven:
   46084312: BENITEZ, JORGE (10 años)
   Más longeva:
   27353752: ALONSO, LORENZO (100 años)
REPORTES: 1 Todas · 2 Mayores · 3 Menores · 4 Por rango · 5 Extremos · 6 Estadísticas · 7 Por
letra · 8 Apellidos repetidos · 9 Volver
Opción: 6
   1987 personas · promedio 54.85 · mínima 10 · máxima 100
REPORTES: 1 Todas · 2 Mayores · 3 Menores · 4 Por rango · 5 Extremos · 6 Estadísticas · 7 Por
letra · 8 Apellidos repetidos · 9 Volver
Opción: 7
Letra: m
   19367737: MUÑOZ, MANUEL (93 años)
   22915455: CAMPOS, MANUEL (14 años)
   27732474: MOYA, MANUEL (41 años)
   47692269: ALVAREZ, MANUEL (24 años)
   59698105: CARMONA, MANUEL (90 años)
   63435960: NUÑEZ, MANUEL (32 años)
   65676280: MENDEZ, MANUEL (50 años)
   87723130: GUERRERO, MANUEL (91 años)
   90308839: LORENZO, MANUEL (42 años)
   99501554: RODRIGUEZ, MANUEL (72 años)
   … y 367 más
   Total: 377
REPORTES: 1 Todas · 2 Mayores · 3 Menores · 4 Por rango · 5 Extremos · 6 Estadísticas · 7 Por
```

```
letra · 8 Apellidos repetidos · 9 Volver
Opción: 8
   MORENO: 33
   CRUZ: 29
   MOLINA: 29
   ORTIZ: 27
   SERRANO: 27
   ROMERO: 26
   RUBIO: 26
   SANCHEZ: 26
   SANTOS: 26
   FUENTES: 25
   100 apellidos se repiten
REPORTES: 1 Todas · 2 Mayores · 3 Menores · 4 Por rango · 5 Extremos · 6 Estadísticas · 7 Por
letra · 8 Apellidos repetidos · 9 Volver
Opción: 9
PERSONAS: 1 Agregar · 2 Cumpleaños · 3 Modificar · 4 Eliminar · 5 Buscar · 6 Reportes · 7 Salir
Opción: 4
Documento: 99000001
   99000001: FERREYRA, LUCÍA (25 años)
¿Confirma la eliminación? (s/n): n
   No se eliminó
PERSONAS: 1 Agregar · 2 Cumpleaños · 3 Modificar · 4 Eliminar · 5 Buscar · 6 Reportes · 7 Salir
Opción: 4
Documento: 99000001
   99000001: FERREYRA, LUCÍA (25 años)
¿Confirma la eliminación? (s/n): s
   Persona eliminada
PERSONAS: 1 Agregar · 2 Cumpleaños · 3 Modificar · 4 Eliminar · 5 Buscar · 6 Reportes · 7 Salir
Opción: 5
Documento: 99000001
   No existe esa persona
PERSONAS: 1 Agregar · 2 Cumpleaños · 3 Modificar · 4 Eliminar · 5 Buscar · 6 Reportes · 7 Salir
Opción: 7
```

Para tener en cuenta

- La prueba agrega a una persona, la modifica y al final la elimina: la base queda como estaba.
- El nombre y el apellido se guardan en mayúsculas, como el resto de la base.